PHYS 4027 / 5027 · FALL 2026 · LECTURE 14

Matrices with Structure, and QR Decomposition

Prof. Alex Chen

Department of Physics

Washington University in St. Louis

Last time

- Gaussian elimination with partial pivoting gives $\mathbf{PA} = \mathbf{LU}$.
- Factoring costs about $\frac{2}{3}n^3$; each solve with $\mathbf{L}$ and $\mathbf{U}$ afterwards about n^2.
- That does not scale: the 10^6 unknowns of a 2D grid would take about 9 years.

Today: matrices from physics are rarely arbitrary. Their structure makes them far cheaper to solve.

Part I · Tridiagonal systems

Only three diagonals

Tridiagonal matrices

$$\begin{pmatrix} b_0 & c_0 & & & \\ a_1 & b_1 & c_1 & & \\ & a_2 & b_2 & \ddots & \\ & & \ddots & \ddots & c_{n-2} \\ & & & a_{n-1} & b_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{pmatrix} = \begin{pmatrix} r_0 \\ r_1 \\ r_2 \\ \vdots \\ r_{n-1} \end{pmatrix}$$

- Store three vectors $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$ instead of the whole matrix: $3n$ numbers instead of n^2.
- Every point coupled only to its two neighbours: the BVP grid of Lecture 12 ($b_i = -2 - \kappa^2 h^2$, $a_i = c_i = 1$), cubic splines, implicit methods for diffusion.

Elimination stays inside the band

- Below each pivot u_{i-1} there is only one nonzero entry, a_i. So each column needs one multiplier, l_i, and changes only the next row.
- That row's super-diagonal c_i is untouched, since the entry above it in row $i-1$ is zero. The factors are

$$\mathbf{A} = \begin{pmatrix} 1 & & & \\ l_1 & 1 & & \\ & \ddots & \ddots & \\ & & l_{n-1} & 1 \end{pmatrix} \begin{pmatrix} u_0 & c_0 & & \\ & u_1 & \ddots & \\ & & \ddots & c_{n-2} \\ & & & u_{n-1} \end{pmatrix}$$

$\mathbf{L}$ and $\mathbf{U}$ are as sparse as $\mathbf{A}$. Nothing fills in.

The recipe (Thomas algorithm)

Start with $u_0 = b_0$, $y_0 = r_0$. Then, top to bottom, for $i = 1, \ldots, n-1$:

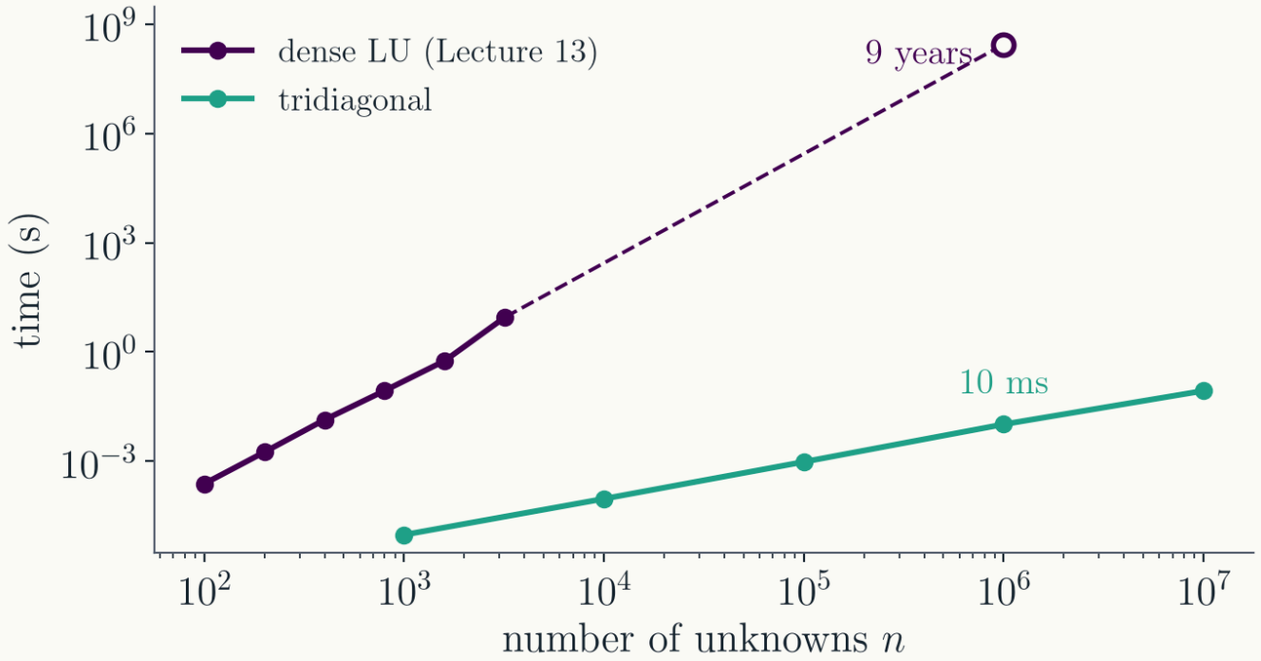
$$l_i = \frac{a_i}{u_{i-1}}, \qquad u_i = b_i - l_i\, c_{i-1}, \qquad y_i = r_i - l_i\, y_{i-1}$$

Then back substitution, bottom to top:

$$x_{n-1} = \frac{y_{n-1}}{u_{n-1}}, \qquad x_i = \frac{y_i - c_i\, x_{i+1}}{u_i}, \quad i = n-2, \ldots, 0$$

The first loop is elimination and forward substitution together; the second is back substitution. About $8n$ operations in all, and about ten lines of C++.

From years to milliseconds



The same matrix, solved two ways, with our own C++ code.

Dense LU at $n = 10^6$: about 9 years (extrapolated).

Tridiagonal at $n = 10^6$: 10 ms. At 10^7: 86 ms.

The time grows like n: twice the unknowns, twice the time.

What about pivoting?

- The recipe never swaps rows. A swap would push entries outside the three diagonals.
- It is safe when the matrix is *diagonally dominant*: each diagonal entry at least as large as the rest of its row,

$$|b_i| \geq |a_i| + |c_i|,$$

  with $>$ in at least one row. Then no u_i can become small.
- Matrices from grids usually are. Lecture 12: $|-2 - \kappa^2 h^2| \geq 1 + 1$, with $>$ in the first and last rows.
- If not, use a banded LU with pivoting (LAPACK's `gbsv`, behind `solve_banded`).

Banded matrices

- A matrix with p nonzero diagonals on each side of the main one. Elimination again stays inside the band: about np^2 operations, storage np.
- A 2D grid of $N \times N$ points, numbered row by row: the neighbours above and below are N places away in the numbering, so $p = N$.
- $N = 1000$: $n = 10^6$ unknowns, about $np^2 = 10^{12}$ operations. Of order ten minutes, instead of 9 years.

Structure turned years into minutes. Sparse direct solvers and iterative methods go further still.

Part II · Cholesky decomposition

Symmetric positive definite matrices

Symmetric positive definite

- **Symmetric:** $a_{ij} = a_{ji}$, so $\mathbf{A}^T = \mathbf{A}$.
- **Positive definite:** $\mathbf{v}^T \mathbf{A} \mathbf{v} > 0$ for every $\mathbf{v} \neq 0$. Equivalently, all eigenvalues are positive.

Physics is full of them, because they are energies:

- Coupled oscillators: the potential energy $\frac{1}{2}\mathbf{x}^T \mathbf{K} \mathbf{x} > 0$ for any displacement, so the spring matrix $\mathbf{K}$ is positive definite.
- Covariance matrices of noise or of fitted parameters; $\mathbf{A}^T \mathbf{A}$ in least squares (Lecture 16).

$\mathbf{A} = \mathbf{L}\mathbf{L}^T$: a 2×2 example

For symmetric positive definite $\mathbf{A}$, look for $\mathbf{U} = \mathbf{L}^T$:

$$\begin{pmatrix} 4 & 2 \\ 2 & 5 \end{pmatrix} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11} l_{21} \\ l_{11} l_{21} & l_{21}^2 + l_{22}^2 \end{pmatrix}$$

Match the entries one at a time:

$$l_{11} = \sqrt{4} = 2, \qquad l_{21} = \frac{2}{l_{11}} = 1, \qquad l_{22} = \sqrt{5 - l_{21}^2} = 2$$

So $\mathbf{L} = \begin{pmatrix} 2 & 0 \\ 1 & 2 \end{pmatrix}$: a lower-triangular "square root" of $\mathbf{A}$.

The recipe

The same matching, column by column, $i = 1, \dots, n$:

$$l_{ii} = \left(a_{ii} - \sum_{k<i} l_{ik}^2 \right)^{1/2}$$

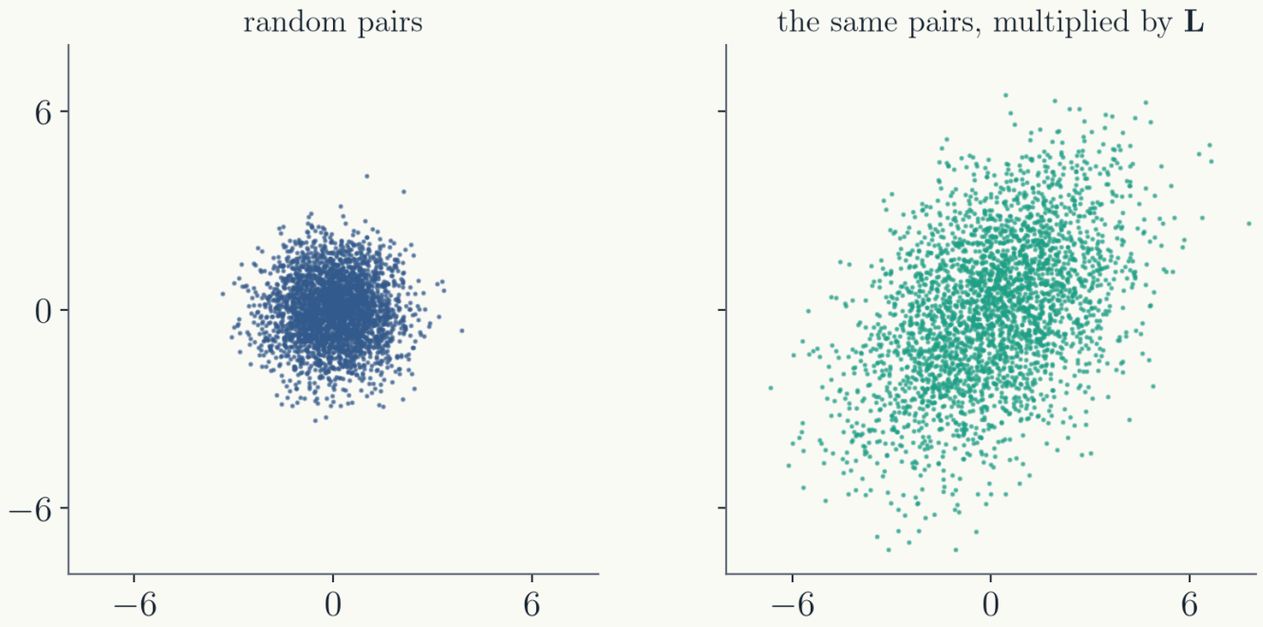
$$l_{ji} = \frac{1}{l_{ii}} \left(a_{ji} - \sum_{k<i} l_{jk}\, l_{ik} \right), \qquad j > i$$

Then solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ as with LU: forward substitution with $\mathbf{L}$, back substitution with $\mathbf{L}^T$.

Why use Cholesky

- **Half the work** of LU, about $\frac{1}{3}n^3$, and half the storage: only $\mathbf{L}$ is kept.
- **No pivoting needed.** It is stable for every symmetric positive definite matrix.
- **A free test.** If $\mathbf{A}$ is not positive definite, a square root of a negative number appears. Attempting Cholesky is the cheapest way to check.

A preview: random numbers with a shape



Later in the semester we will generate random numbers and use them in simulations.

Left: pairs of random numbers with nothing to do with each other form a round cloud.

Multiply every pair by the $\mathbf{L}$ of our 2×2 example, and the cloud stretches and tilts: the two numbers now tend to rise and fall together.

Real noise often looks like the right panel. Cholesky is how simulations produce it, and we will come back to it with random numbers and Monte Carlo methods.

Tell the library what you know

- `numpy.linalg.solve` always uses general LU with partial pivoting (LAPACK's `gesv`).
- `scipy.linalg.solve(A, b, assume_a="pos")` uses Cholesky, for symmetric positive definite $\mathbf{A}$. `scipy.linalg.solve_banded` handles tridiagonal and banded matrices.
- Eigen (C++): `PartialPivLU` for general matrices, `LLT` for Cholesky, and a separate module for sparse matrices.

The library cannot guess the structure. If you do not tell it, it pays the full n^3.

Part III · QR decomposition

Orthogonal times triangular

$\mathbf{A} = \mathbf{QR}$

- $\mathbf{R}$ is upper triangular. $\mathbf{Q}$ is *orthogonal*: its columns are unit vectors, perpendicular to each other, so $\mathbf{Q}^T\mathbf{Q} = \mathbf{I}$.
- For the matrix we will use below:

$$\underbrace{\begin{pmatrix} 4 & 2 & 1 \\ 2 & 1 & 0 \\ 1 & 0 & 3 \end{pmatrix}}_{\mathbf{A}} = \underbrace{\begin{pmatrix} -0.873 & -0.195 & -0.447 \\ -0.436 & -0.098 & 0.894 \\ -0.218 & 0.976 & 0 \end{pmatrix}}_{\mathbf{Q}} \underbrace{\begin{pmatrix} -4.583 & -2.182 & -1.528 \\ 0 & -0.488 & 2.733 \\ 0 & 0 & -0.447 \end{pmatrix}}_{\mathbf{R}}$$

The first column of $\mathbf{Q}$ is the first column of $\mathbf{A}$, normalized: $-(4, 2, 1)/\sqrt{21}$. How to compute $\mathbf{Q}$ and $\mathbf{R}$ in general: next lecture.

Why orthogonal matrices are safe

- **The inverse is free:** $\mathbf{Q}^{-1} = \mathbf{Q}^T$.
- **Lengths are preserved:** $|\mathbf{Q}\mathbf{x}| = |\mathbf{x}|$. A rotation or reflection does not stretch anything, so it cannot amplify rounding errors.
- **Solving:** $\mathbf{Q}\mathbf{R}\mathbf{x} = \mathbf{b}$ becomes $\mathbf{R}\mathbf{x} = \mathbf{Q}^T\mathbf{b}$, then back substitution.
- **The price:** about twice the work of LU. For square systems LU is the default. QR is the tool for rectangular matrices (least squares, Lecture 16) and for eigenvalues.

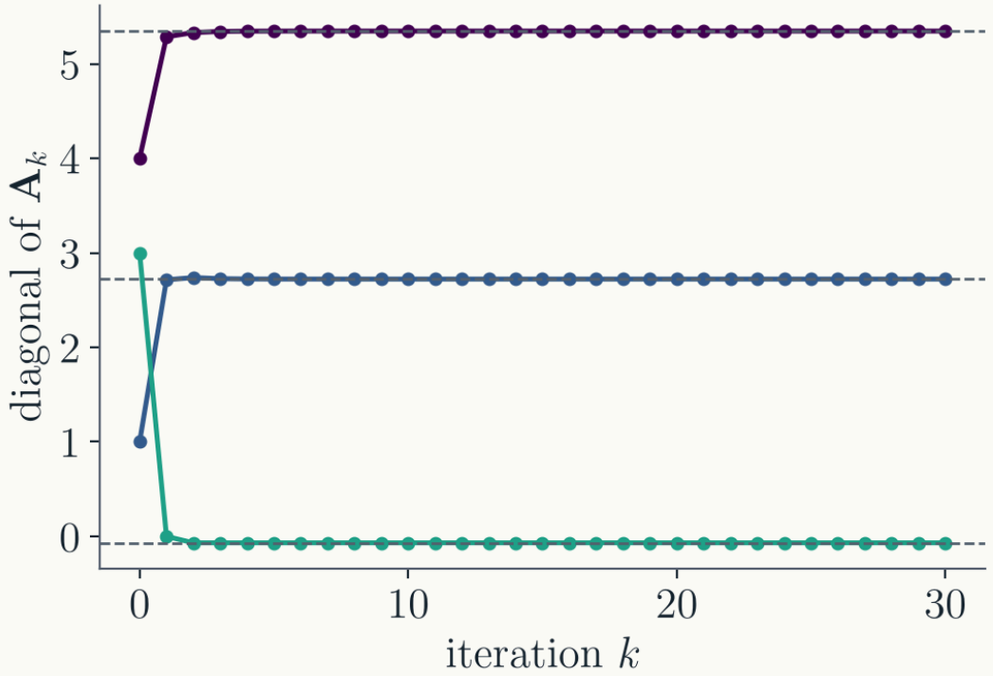
Eigenvalues from QR

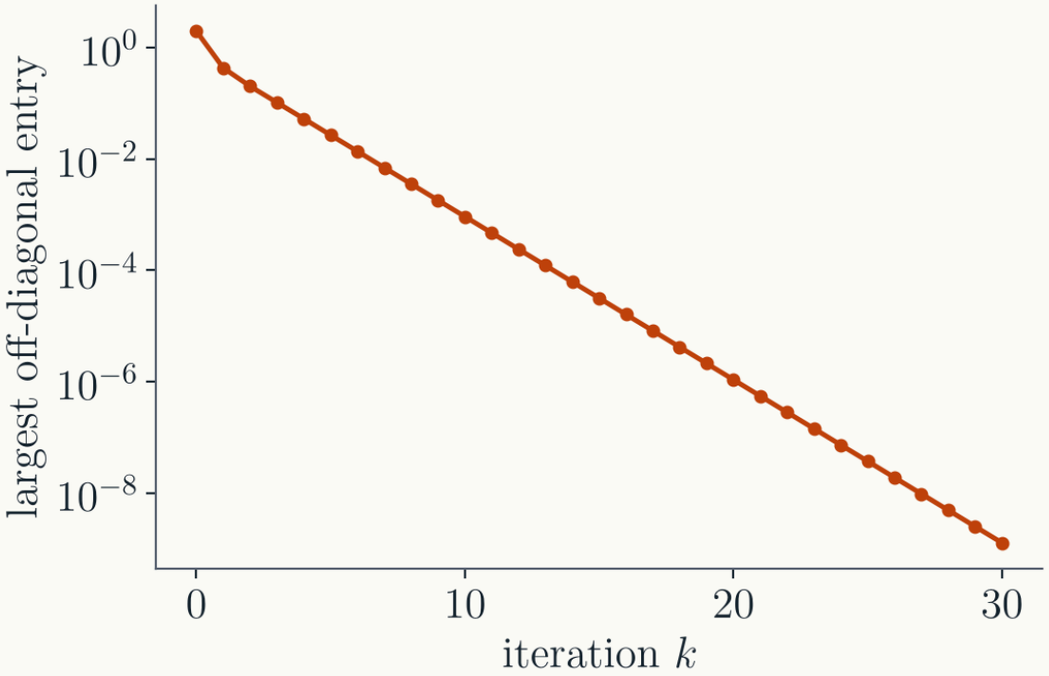
Start with $\mathbf{A}_0 = \mathbf{A}$, and repeat:

$$\text{factor } \mathbf{A}_k = \mathbf{Q}_k \mathbf{R}_k, \qquad \text{multiply in reverse: } \mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k$$

- Since $\mathbf{R}_k = \mathbf{Q}_k^T \mathbf{A}_k$: $\mathbf{A}_{k+1} = \mathbf{Q}_k^T \mathbf{A}_k \mathbf{Q}_k$. This is a change of basis, so $\mathbf{A}_{k+1}$ has the same eigenvalues as $\mathbf{A}_k$, and so as $\mathbf{A}$.
- The iterates become triangular (diagonal, if $\mathbf{A}$ is symmetric). The eigenvalues of a triangular matrix are its diagonal entries.

The QR iteration on our matrix





- The diagonal settles on the eigenvalues 5.346, 2.722, -0.069 within a few iterations.
- The off-diagonal entries shrink by about a factor of 2 per iteration, the ratio $2.72/5.35$ of two eigenvalues: 10^{-3} after 10 iterations, 10^{-9} after 30. Close eigenvalues would mean slow convergence.

What the libraries actually do

- First, bring $\mathbf{A}$ to tridiagonal form (if symmetric) with orthogonal transformations. Each QR step then costs only about n operations.
- Then QR iterations with *shifts*, which make convergence fast even for close eigenvalues: a few iterations per eigenvalue.
- This is `numpy.linalg.eigh` (symmetric) and `eig` (general): about n^3 in total.
- The particle in a box on a grid is already tridiagonal: `scipy.linalg.eigh_tridiagonal` skips the first step.

Summary

the matrix	the method	the cost
general, square	LU with partial pivoting	$\frac{2}{3}n^3$
tridiagonal	Thomas algorithm	about $8n$
banded, width p	banded LU	about np^2
symmetric positive definite	Cholesky, $\mathbf{L}\mathbf{L}^T$	$\frac{1}{3}n^3$
any shape; eigenvalues	QR	$\frac{4}{3}n^3$ (square)

Next time: how to compute $\mathbf{Q}$ and $\mathbf{R}$, and the singular value decomposition.